

- 1) A Genetic Algorithm is applied to maximize the function $f(x) = x^2$ using an initial population consisting of the values 2, 4, 6, and 8. Determine the fitness value for each individual in the population and select the best candidates based on their fitness scores. Perform a crossover operation by exchanging the least significant bits of the selected individuals to generate new offspring. Additionally, explain the role of mutation in improving the diversity and effectiveness of the solution in Genetic Algorithms.

INPUT CODE:

```
population = [2, 4, 6, 8]
print("INITIAL POPULATION")
print("-----")
print(population)
fitness = [x**2 for x in population]
print("\nFITNESS VALUES")
print("-----")
for x, f in zip(population, fitness):
    print(f"Individual = {x}    Fitness = {f}")
combined = list(zip(population, fitness))
combined.sort(key=lambda x: x[1], reverse=True)
selected = combined[:2]
print("\nSELECTED INDIVIDUALS")
print("-----")
for x, f in selected:
    print(f"Individual = {x}    Fitness = {f}")
parent1 = selected[0][0]
parent2 = selected[1][0]
bin1 = format(parent1, '04b')
bin2 = format(parent2, '04b')
print("\nBINARY REPRESENTATION")
print("-----")
print(f"Parent 1 ({parent1}) = {bin1}")
print(f"Parent 2 ({parent2}) = {bin2}")
print("\nAFTER Crossover")
print("-----")
print(f"Offspring 1 = {offspring1} = {off1_decimal}")
print(f"Offspring 2 = {offspring2} = {off2_decimal}")
mutated = list(offspring1)
mutated[-1] = '1' if mutated[-1] == '0' else '0'
mutated_fitness = mutated_decimal ** 2
print("\nAFTER MUTATION")
print("-----")
print(f"Mutated Offspring = {mutated}")
print(f"Decimal Value = {mutated_decimal}")
print(f"Fitness = {mutated_fitness}")
```

OUTPUT :

INITIAL POPULATION

[2, 4, 6, 8]

FITNESS VALUES

Individual = 2 Fitness = 4

Individual = 4 Fitness = 16

Individual = 6 Fitness = 36

Individual = 8 Fitness = 64

SELECTED INDIVIDUALS

Individual = 8 Fitness = 64

Individual = 6 Fitness = 36

BINARY REPRESENTATION

Parent 1 (8) = 1000

Parent 2 (6) = 0110

AFTER Crossover

Offspring 1 = 1000 = 8

Offspring 2 = 0110 = 6

AFTER MUTATION

Mutated Offspring = 1001

Decimal Value = 9

Fitness = 81

- 2) A perceptron model is used to classify whether a customer will purchase a product based on two features: advertisement exposure and income level. The weights are 0.5 and 0.3, and the bias is -0.4 . For a customer with advertisement exposure = 2 and income level = 1, compute the net input to the perceptron and apply the step activation function. Determine whether the customer makes a purchase or not.

INPUT CODE:

```
# Perceptron Model Example
# Input Features
advertisement_exposure = 2
income_level = 1
# Weights
w1 = 0.5
w2 = 0.3
# Bias
bias = -0.4
# Calculate Net Input
net_input = (w1 * advertisement_exposure) + (w2 * income_level) + bias
print("PERCEPTRON MODEL")
print("-----")
print(f"Advertisement Exposure = {advertisement_exposure}")
print(f"Income Level = {income_level}")
print(f"Weight 1 = {w1}")
print(f"Weight 2 = {w2}")
print(f"Bias = {bias}")
print("\nNet Input Calculation")
print("-----")
print(f"Net Input = ({w1} × {advertisement_exposure}) + ({w2} × {income_level})")
print(f"Net Input = {net_input}")
# Step Activation Function
if net_input >= 0:
    output = 1
    decision = "Customer will Purchase the Product"
else:
    output = 0
    decision = "Customer will NOT Purchase the Product"
print("\nStep Activation Function")
print("-----")
print(f"Output = {output}")
print("\nFinal Decision")
print("-----")
print(decision)
```

Output :

```
PERCEPTRON MODEL
-----
Advertisement Exposure = 2
Income Level = 1
Weight 1 = 0.5
Weight 2 = 0.3
Bias = -0.4

Net Input Calculation
-----
Net Input =  $(0.5 \times 2) + (0.3 \times 1) + (-0.4)$ 
Net Input = 0.9

Step Activation Function
-----
Output = 1

Final Decision
-----
Customer will Purchase the Product
```